

操作系统研讨课

Course: B0911011Y

卢天越

2026 春季学期



Project 1 BootLoader

2026.3.9



今日课程内容

- 介绍Project 1
- 答疑



Start-code

- 每个Project有一个start-code
 - 作为代码框架
 - 包含一些已经写好的代码和代码功能的提示



代码框架

- 仿照Linux代码风格
- 每个文件夹对应不同用处
- P1主要用到的：
 - arch文件夹：汇编代码
 - Init文件夹：初始化代码

arch/riscv

include

init

kernel/loader

libs

test/test_project1

tiny_libc/include

tools



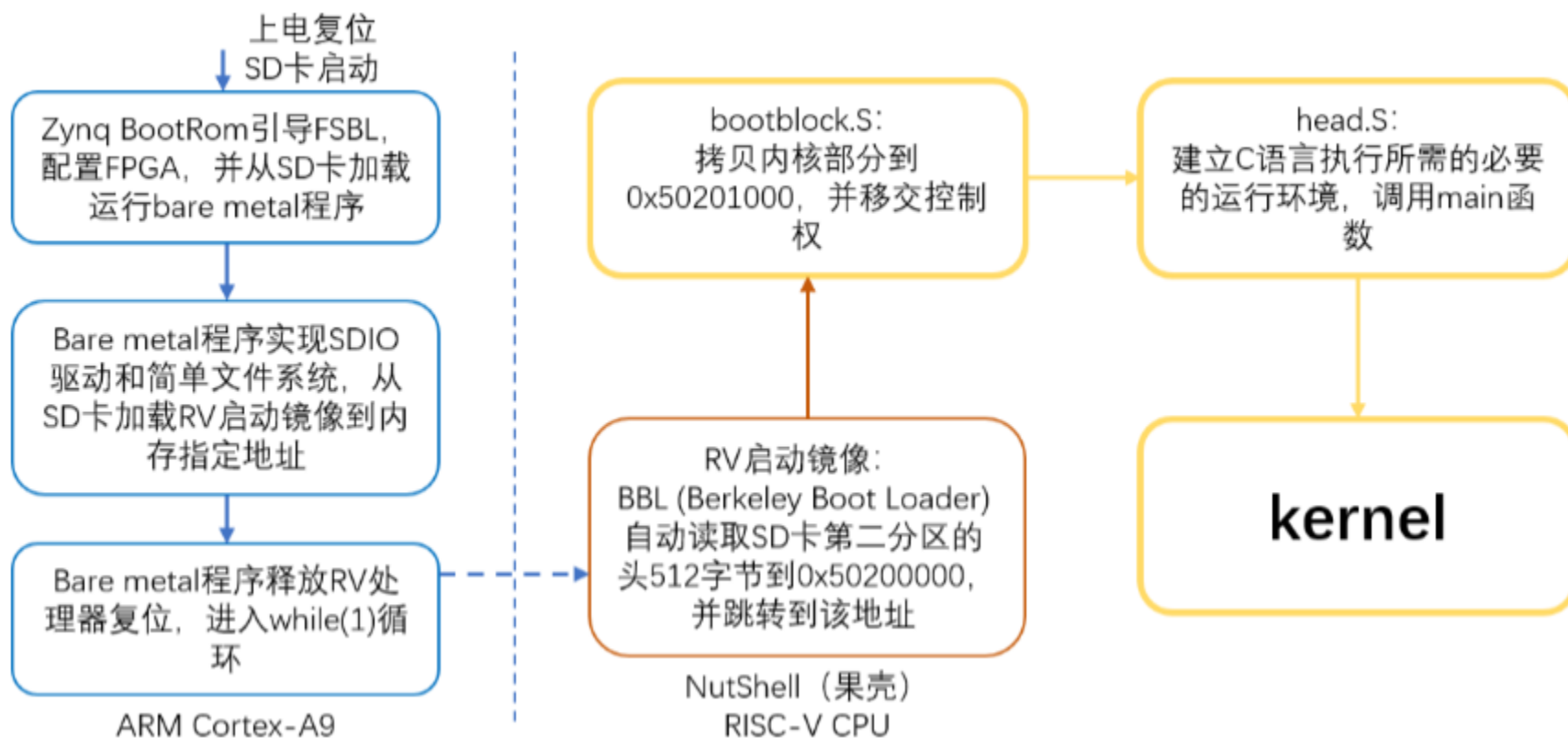
Project1 引导启动

- 对于一个裸板子 (bare metal) , 程序从哪里开始运行?
 - ROM, BIOS
 - BIOS会做最基本的设置工作, 然后跳转到操作系统代码
 - 只有512字节的数据
 - 称为BootLoader, 负责加载其他的操作系统代码
 - Boot: n. 靴子; v. 启动



课程开发板的引导启动

- 关注黄色部分



bootblock.S

- 要做的3个事情

```
19 // TODO: [p1-task1] call BIOS to print string "It's bootblock!"
```

1、打印一个字符串

```
22 // TODO: [p1-task2] call BIOS to read kernel in SD card
```

2、从SD卡中加载操作系统内核的其他部分

```
25 // TODO: [p1-task4] load task-related arguments and pass them to kernel
```

```
28 // TODO: [p1-task2] jump to kernel to start UCAS-OS
```

3、跳转到内核开始运行



如何在BootLoader里打印

- 一些BIOS中已经提供好的接口函数
 - 提供一些系统服务，包括打印
- 使用统一的入口地址
 - 可以理解为函数入口
 - 函数参数放在a1开始的寄存器中，要调用的函数编号放在a7寄存器中

```
9  // BIOS function entry (jump here to use BIOS APIs)
10 .equ bios_func_entry, 0x50150000
```



如何读取SD卡中的OS代码

- 依然使用bios_func_entry的服务函数读取
- 三个参数：
 - 1、内存地址：要放到内存的哪里？
 - 2、扇区数：操作系统代码有多大？
 - 3、起始扇区号：操作系统代码在哪里？



内存空间

- BIOS把BootLoader放到了哪里？
 - 0x50200000, 一个写死的地址
 - BIOS (bbl) 自动放的
 - 内核放到它的后面, 0x50201000
 - 需要你来放
 - 可以认为也是写死的地址。和编译有关, 有兴趣的同学可以参考Makefile和riscv.ld里面的内容



SD卡中的OS代码位置

- SD卡第三个分区中放有你的OS代码
- 第一个扇区是BootLoader
- 从第二个扇区开始是kernel代码
 - Kernel代码的大小?
 - 为了简化, 可以先拷贝足够大的分区即可
 - 从P1的任务4开始要精准的大小



从BootLoader跳转到kernel

- 拷贝结束后，即可跳转到kernel代码运行
- 用什么跳转指令？
 - 考虑到BootLoader的代码不会回来运行了，用直接跳转指令 `j` 即可
- Kernel代码的内存地址在哪里？
 - 拷贝的时候你已经知道了~
- 截止到这里，P1的前两个任务已经完成了



任务1和任务2

- 任务1：完成在BootLoader中的打印
- 任务2：完成在BootLoader中加载kernel并跳转过去，在kernel代码中打印
- 问题：BootLoader是汇编代码，kernel是C代码，跳转过去需要注意什么？
- head.S：设置C语言运行环境
 - 准备好栈（分配栈空间，设置sp寄存器）
 - 清空bss段（对于未初始化的全局变量清零）



注意head.S

- 设置栈
 - 注意分配的栈空间，不要和程序代码冲突
- 清理bss段
 - 从ldscript找标号

```
9  #define KERNEL_STACK          0x50500000
10
11  .section ".entry_function", "ax"
12  ENTRY(_start)
13  /* Mask all interrupts */
14  csrw CSR_SIE, zero
15  csrw CSR_SIP, zero
16
17  /* TODO: [p1-task2] clear BSS for flat non-ELF images */
18
19
20  /* TODO: [p1-task2] setup C environment */
```



思考

- 为什么需要一个单独的BootLoader，而不是一次性把kernel所有内容都加载到内存？



任务3：kernel加载运行用户程序

- 跳转到内核之后，我们的目标是可以简单的把用户程序运行起来
 - Kernel可以用和BootLoader类似的方法，把用户程序加载进内存
- 问题是：用户程序在哪、放到哪？
- 回溯到任务1、2：BootLoader、kernel、用户程序，这些不一起编译的东西，是怎么一起放到SD卡的开头并排列好的？



createimage

- 操作系统镜像 (image) 文件

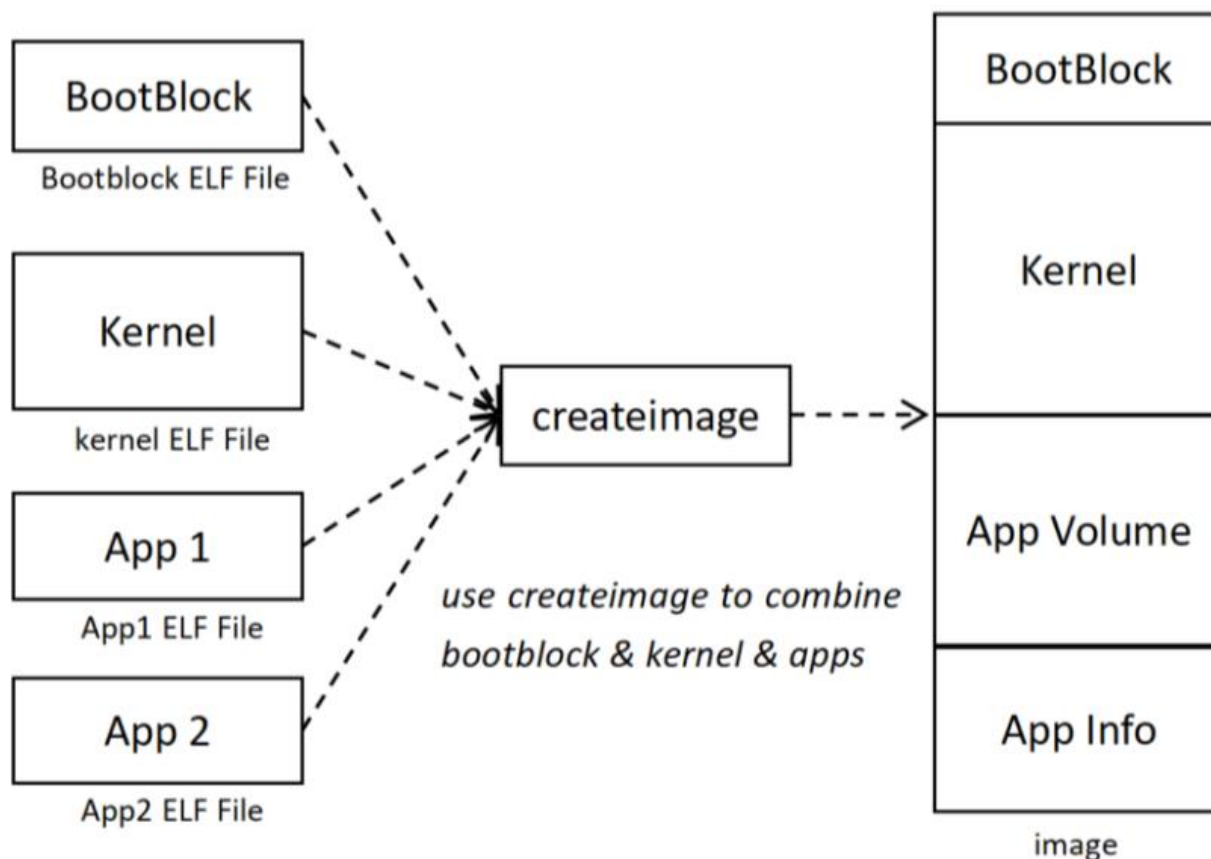
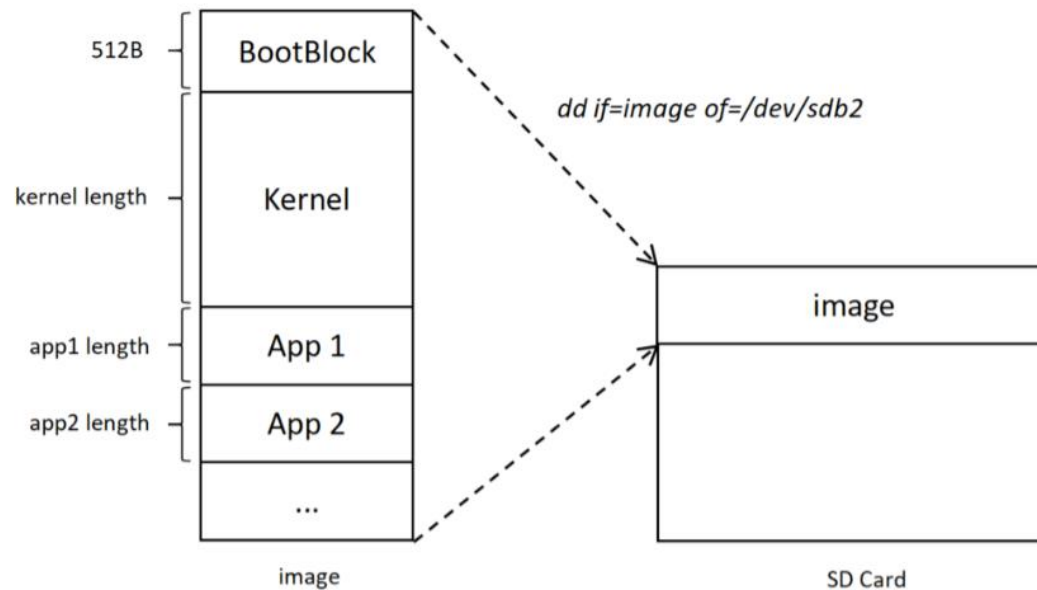


image->SD卡

- 通过镜像文件，我们实现了BootLoader、kernel、用户程序放在SD卡的开头
 - 可以参见Makefile



用户程序在哪

- createimage程序中，我们可以知道kernel、用户程序的大小
- 可以留下一些痕迹在image中，让BootLoader和kernel可以读到
 - 一个可以方便留痕的地方：BootLoader所在扇区的空闲位置（因为BootLoader肯定占不满512字节的大小）
 - BootLoader可以根据这个痕迹知道kernel大小
 - Kernel可以根据这个痕迹知道用户程序的位置



kernel/loader/loader.c

- 任务3：通过输入的task id，得到image中用户程序的位置，放入内存
 - 内存地址：从0x52000000开始
 - 简化：kernel和所有用户程序都占同样多的扇区数，方便计算位置

```
6  uint64_t load_task_img(int taskid)
7  {
8      /**
9       * TODO:
10      * 1. [p1-task3] load task from image via task id, and return its entrypoint
11      * 2. [p1-task4] load task via task name, thus the arg should be 'char *taskname'
12      */
13
14      return 0;
15 }
```

tools/createimage.c

- 为了实现简化：kernel和所有用户程序都占同样多的扇区数，方便计算位置
 - createimage的时候要加上空白（padding）
 - 虽然简化了计算，但是浪费了空间

```
123      /* write padding bytes */
124      /**
125       * TODO:
126       * 1. [p1-task3] do padding so that the kernel and every app program
127       *    occupies the same number of sectors
128       * 2. [p1-task4] only padding bootblock is allowed!
129      */
```



任务4：image文件的紧密排列

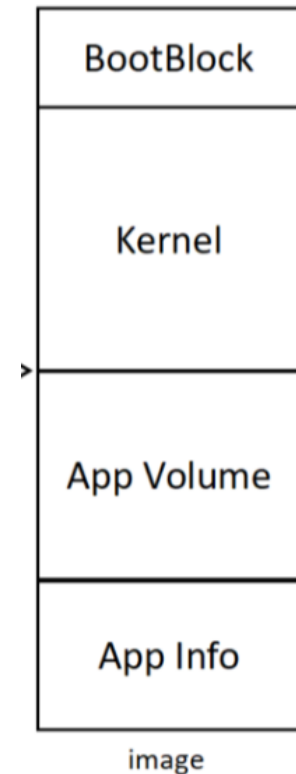
- 如果紧密排列，createimage不用加padding了，但是如何在bootblock里面表示复杂的地址位置呢？
 - 其实bootblock的空白位置放几个地址信息是足够的，但是这样扩展性不好
- 本任务开始为A-core任务
 - A-core验收务必使用-O2编译优化



任务4的提示

- Bootblock的空白位置放一个指针，指向image文件的其他位置
 - 指向一个结构体数组
 - 放各个用户程序的信息

```
20  /* TODO: [p1-task4] design your own task_info_t */
21  typedef struct {
22
23  } task_info_t;
24
```



Kernel中调用bios提供的函数

- Kernel是C语言代码了，如何调用bios提供的函数
 - 嵌入汇编（不推荐）
 - 在init/main.c里面提供的跳转表

```
29 static void init_jmptab(void)
30 {
31     volatile long ((*jmptab))() = (volatile long ((*))())KERNEL_JMPTAB_BASE;
32
33     jmptab[CONSOLE_PUTSTR] = (long (*)(void))port_write;
34     jmptab[CONSOLE_PUTCHAR] = (long (*)(char))port_write_ch;
35     jmptab[CONSOLE_GETCHAR] = (long (*)(void))port_read_ch;
36     jmptab[SD_READ] = (long (*)(void))sd_read;
37 }
```

思考

- 我们用一套数据结构管理了多个用户程序，这和真正的文件系统还有多少不同？



任务5：批处理

- 本任务为C-core任务
- 通过批处理的方式连续运行多个程序，且每个程序的输出作为下一个程序的输入
 - 输出为数字，每个程序都是做一些数学运算
- 注意点：
 - 不能用户程序直接跳转进另一个用户程序
 - 输出输入之间可以约定固定内存地址
 - 详见任务书



总结P1任务

- S-core任务：任务1-3
 - Bootloader打印，加载，跳转
 - Kernel加载指定用户程序并运行用户程序
- A-core任务：任务1-4
 - 在S-core的基础上，要求镜像文件不能有空泡
- C-core任务：任务1-5
 - 完成A-core的同时，实现批处理任务



其他注意事项

- S-core和A-core的测试程序在test/test_project1文件夹下，C-core测试程序需要自己写
- 希望同学们能理解Makefile的内容，特别是跟内存地址相关的部分
- 代码框架只是一个提示，同学们可以根据自己的想法和需求修改start-code内容
- 提供的createimage：只适用任务1、2



其他注意事项

- 先在QEMU模拟器上完成，两周后验收
 - 先在模拟器上验收，通过之后当场发FPGA开发板，在开发板上当场完成后算作P1通过
- 任务书内容比ppt详细，做之前请仔细阅读
- Linux系统的使用：
 - 注意看输出
 - 输命令时多用tab键自动补齐
- Design review需要回答的问题见下页



P1 Design review

- Design review分组下周课前公布
 - 每人单独跟老师介绍
 - 形式不限, ppt、doc、代码均可
- 最好覆盖以下问题:
 - 代码框架中每个文件夹放什么文件
 - 结合内存地址说明从上电到用户程序运行的过程
 - head.S都需要做什么, 为什么要做
 - 镜像文件的结构是什么样的, 从编译到生成镜像文件的过程是什么样的
 - (A-core) 你的task_info_t结构体是如何设计的



谢谢！

下节课：3月20日，P1 design review

注意时间为周五上午2、3、4节

Any questions?

